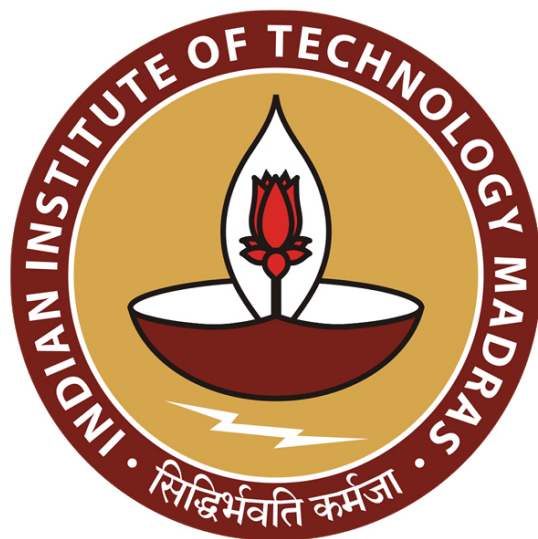


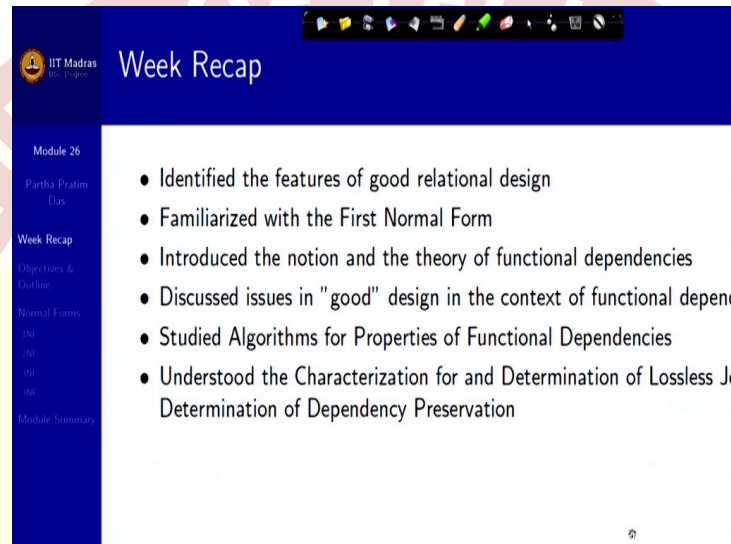


IIT Madras
ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology Kharagpur
Relational Database Design/6: Normal Forms

Welcome to week 6, module 26 of database management systems course in IIT Madras, online BSc program.

(Refer Slide Time: 00:34)

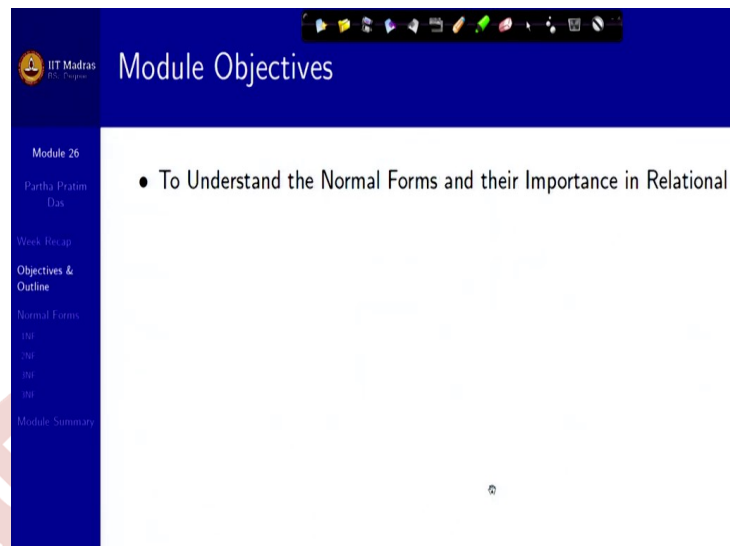


Week Recap

- Identified the features of good relational design
- Familiarized with the First Normal Form
- Introduced the notion and the theory of functional dependencies
- Discussed issues in "good" design in the context of functional dependencies
- Studied Algorithms for Properties of Functional Dependencies
- Understood the Characterization for and Determination of Lossless Join and Determination of Dependency Preservation

We have been discussing about different aspects of the design of relational databases, particularly focusing on a complete mathematical framework based on the dependency theory. And in the whole of last week, we did cover various aspects of it starting from the basic notions of good design at multiple places, the theory of functional dependencies, different properties of functional dependencies, algorithms for functional dependencies and characterization of Lossless Join and dependency preservation, which are key properties for having a good decomposed relation.

(Refer Slide Time: 01:28)

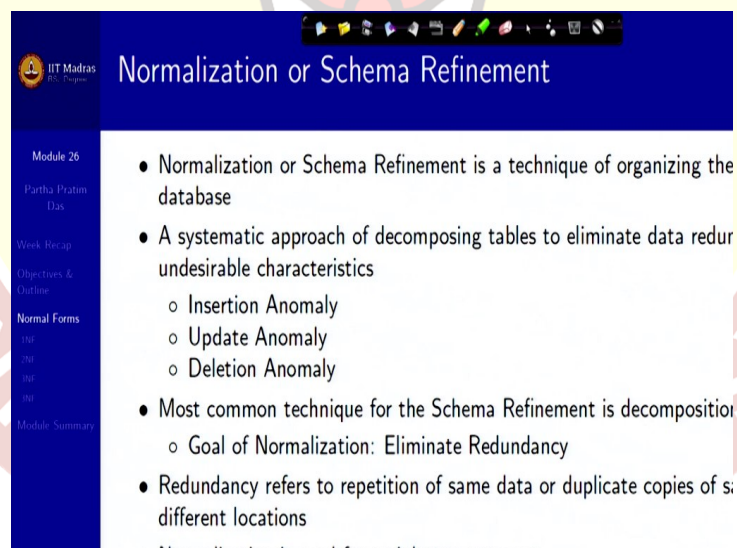


The slide is titled "Module Objectives" and is part of a presentation from IIT Madras. The left sidebar shows the navigation menu with "Module 26" selected. The main content area lists the following objective:

- To Understand the Normal Forms and their Importance in Relational

In this week 2, we will continue on that developing that. And in the current module, I will just take you through some of the initial normal forms of representation of relational schemas. And how are they important in our design.

(Refer Slide Time: 01:57)



The slide is titled "Normalization or Schema Refinement" and is part of a presentation from IIT Madras. The left sidebar shows the navigation menu with "Module 26" selected. The main content area lists the following points:

- Normalization or Schema Refinement is a technique of organizing the database
- A systematic approach of decomposing tables to eliminate data redundancy and undesirable characteristics
 - Insertion Anomaly
 - Update Anomaly
 - Deletion Anomaly
- Most common technique for the Schema Refinement is decomposition
 - Goal of Normalization: Eliminate Redundancy
- Redundancy refers to repetition of same data or duplicate copies of data at different locations

So, we have talked about a little about normal forms earlier when we talked about the first normal form. The whole idea is a, it is kind of a refinement technique or a standardization technique of organizing data in a database, which satisfies some of the key properties. Now, we have already discussed that there is a, the reason we need to normalize is normalization actually gives us good decomposition. Good decomposition gives us minimization of redundancy.

Minimization of redundancy means, removal or minimization of anomalies. So, that is the basic structure of why we will have to normalize or why we will have to decompose we have already discussed. So, we can say in one word that the goal of normalization is to eliminate redundancy, redundancy of data. And also to ensure, that the data dependency is done in a sensible manner.

(Refer Slide Time: 03:13)

IIT Madras
For Engineers

Module 26

Partha Pratim Das

Work Recap

Objectives & Outline

Normal Forms

1NF

2NF

3NF

4NF

Module Summary

Anomalies

a) **Update Anomaly:** Employee 519 is shown as having different addresses on different records

Employee ID	Employee Address	Skill
426	87 Sycamore Grove	Typing
426	87 Sycamore Grove	Shorthand
519	34 Chestnut Street	Public Speaking
519	36 Walnut Avenue	Carpentry

Resolution: Decompose the Schema

a) *Update:* (ID, Address), (ID, Skill)

b) **Insertion Anomaly:** Until member, Dr. Newsome, is at least one course, his de

Faculty ID	Faculty Name	Facu
389	Dr. Giddens	10-Fe
407	Dr. Saperstein	19-Apr
407	Dr. Saperstein	19-Apr
424	Dr. Newsome	29-

recorded

c) **Deletion Anomaly:** All ir Dr. Giddens is lost if he t to be assigned to any cou

Faculty ID	Faculty Name	Faculty Hire Date
------------	--------------	-------------------

Here are examples of anomalies, we have already discussed this. It is a, these are little different examples. So, you can just go through and recap on the notions of anomalies or you can go back to the earlier module and check on that. There is possibility of insertion anomaly, deletion anomaly and update anomaly. And we will have to avoid all of that.

(Refer Slide Time: 03:40)

IIT Madras
For Engineers

Module 26

Partha Pratim Das

Work Recap

Objectives & Outline

Normal Forms

1NF

2NF

3NF

4NF

Module Summary

Desirable Properties of Decomposition

- Lossless Join Decomposition Property
 - It should be possible to reconstruct the original table
- Dependency Preserving Property
 - No functional dependency (or other constraints) should get violated

ORIGINAL SCHEMA

1 NF

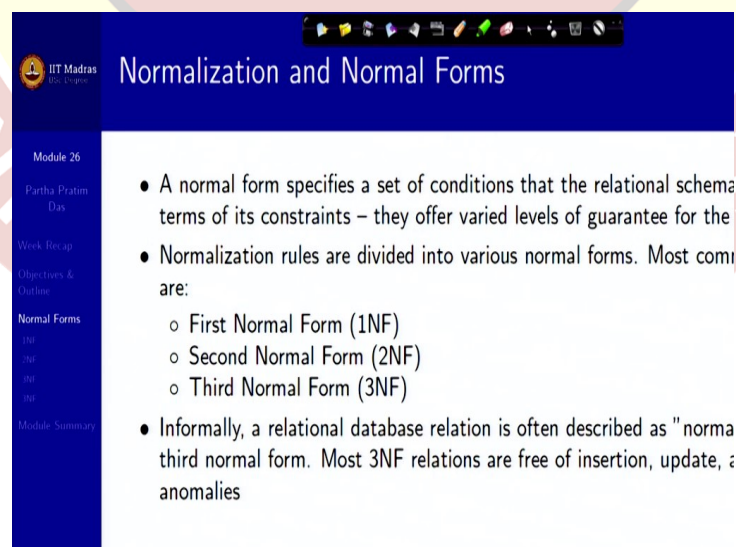
2 NF

So, the process of doing that in decomposition as this tree shows, that you start with the original schema, and you decompose it into smaller schemas. And you do it in a way, that the smaller schema satisfies some of the desirable properties. But, it may not be enough for a particular relation. And if it is not enough, if you still have redundancy, then you may have to put additional conditions, additional properties and decompose it further, and this goes on.

So, in this tree we have shown up to the third normal form, the, you start with the original schema. If all relations in that is not in the first normal form, you put them in the, you decompose and put them in the first normal form. We have shown examples of how to do this. If they are not in the first normal, if they are in the first normal form and still have redundancies, you would consider putting them in the second normal form. We will discuss what it is.

If there is redundancy still, you will put it in the third normal form and so on. Now, as you decompose, it is critical that you maintain two properties. The first property is absolutely essential that you have Lossless Join. In the modules last week, we have discussed in depth about it, that it should be possible to reconstruct original data. And the second property is dependency preservation that is, in terms of the decomposed relations, I must still be able to check for all the functional dependencies or all the different forms of dependencies without actually computing a join of the data we have discussed this as well.

(Refer Slide Time: 05:41)



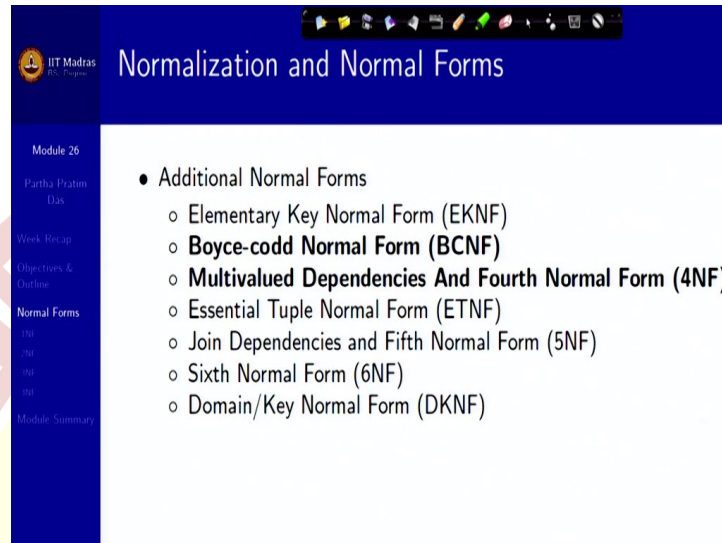
The screenshot shows a presentation slide from IIT Madras. The title is 'Normalization and Normal Forms'. The slide is part of Module 26, 'Partha Pratim Das'. The left sidebar lists the following items: 'Work Recap', 'Objectives & Outline', 'Normal Forms', and 'Module Summary'. The main content area contains the following bullet points:

- A normal form specifies a set of conditions that the relational schema terms of its constraints – they offer varied levels of guarantee for the
- Normalization rules are divided into various normal forms. Most common are:
 - First Normal Form (1NF)
 - Second Normal Form (2NF)
 - Third Normal Form (3NF)
- Informally, a relational database relation is often described as "normal third normal form". Most 3NF relations are free of insertion, update, & anomalies

So, with this, we go into introducing the normal forms, specifies a set of conditions that the relational schema must satisfy in terms of its constraints. The common normal forms are first normal form; you already know that. You have second normal form, you have third normal

form and so on. Informally, it is more often that the designers would use the third normal form to normalize a database. So, if you hear that this is a normalized database and nothing specifically mentioned, then you will know that this is a third normal form normalization.

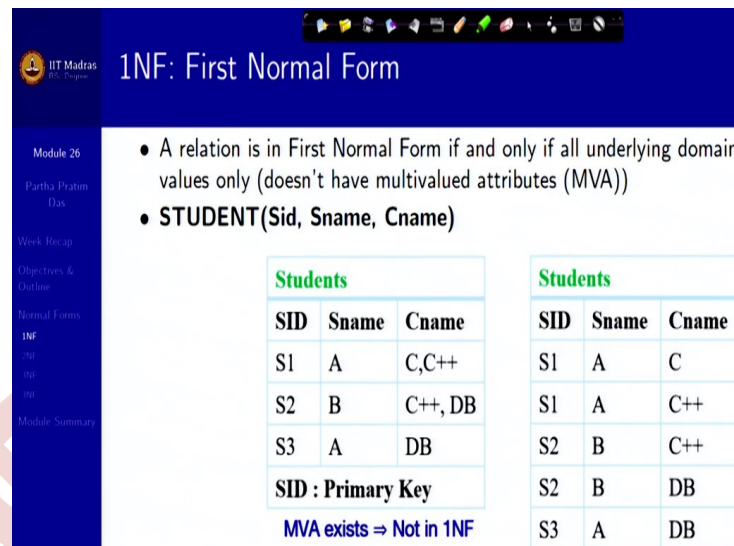
(Refer Slide Time: 06:24)



But these are not the only forms of only normal forms, you have already heard, seen Boyce-codd or BCNF normal form. Here are some more, like elementary key normal form, Boyce-codd you have seen multivalued dependency and fourth normal form, fifth normal form, sixth normal form and so on. Obviously, we will not have time to discuss all of them, but the highlighted ones BCNF and 4NF also we will discuss in a later module.

Because it needs some more of the functional dependency theory, particularly the 4NF. And first second and third normal form can be understood totally in terms of the functional dependency theory, that you have already learned.

(Refer Slide Time: 07:10)



1NF: First Normal Form

- A relation is in First Normal Form if and only if all underlying domain values only (doesn't have multivalued attributes (MVA))
- **STUDENT(Sid, Sname, Cname)**

SID	Sname	Cname
S1	A	C,C++
S2	B	C++, DB
S3	A	DB

SID : Primary Key
MVA exists ⇒ Not in 1NF

SID	Sname	Cname
S1	A	C
S1	A	C++
S2	B	C++
S2	B	DB
S3	A	DB

The first normal form, what is the basic criteria? That underlying domain contains atomic values. Now, we, we had been very explicit to say that every value has to be atomic. And no attribute can be multiple valued. In generality, often these two are taken together to mean atomic value, that if a value is atomic, it means that it is indivisible. So, if there can be multiple values in the same field, then they are seen as if these are different components of the value and therefore not atomic.

So, more often people do not mention the two conditions separately, people will say that it contains atomic values. So, there is a relation here, which on the left has multivalued attributes, the course name, the courses that the students are attending. And on the right, you have removed that and made that into the, into a 1NF form even though we know that this is not very desirable. Because your primary key is now changing from SID to SID and Sname. We have discussed this.

(Refer Slide Time: 08:34)

IIT Madras

1NF (2): Possible Redundancy

Module 26

Partha Pratim Das

Week Recap

Objectives & Outline

Normal Forms

1NF

2NF

3NF

Module Summary

- Example: Supplier(SID, Status, City, PID, Qty)

Supplier:

SID	Status	City	PID	Qty
S1	30	Delhi	P1	100
S1	30	Delhi	P2	125
S1	30	Delhi	P3	200
S1	30	Delhi	P4	130
S2	10	Karnal	P1	115
S2	10	Karnal	P2	250
S3	40	Rohtak	P1	245
S4	30	Delhi	P4	300
S4	30	Delhi	P5	315

Key : (SID, PID)

Drawbacks:

- Deletion Anomaly: If we delete <S3,40,Rohtak,P1,245>, then we lose the information that
- Insertion Anomaly: We cannot insert a Supplier S5 located in Karnal until S5 supplies at least

FD

Now, let us take a different example, we were discussing this example earlier too, in terms of Lossless Join. So, we have a supplier relationship with supplier ID, status, city, product ID and quantity. Every city has a status and on the right, what you see here, what you see here is the, these are the FDs basically. So, we say that SID uniquely determined city, supplier given the supplier you know the city, given the city you know its status.

Given the SID and PID, supplier ID and product ID you know the quantity. So, this is when instead of writing it down, this is just shown in terms of a diagram, people often do that. Now naturally this, this representation this is in 1NF. Because it does not, it has only atomic values, but it has several redundancy and possible problems. For example, if I delete this, record S3 then the information that Rohtak is a city and its status is 40 is going to get lost.

Because that is the only entry for Rohtak. If we, we cannot insert a supplier S5 who is located in Karnal until the supplier has supplied at least one part. Because you can see that part is a, PID is a attribute which is part of the key. So, we have insertion anomalies, if a supplier S1 moves from Delhi to Kanpur, then it is difficult because there are already four instances, there could be more. You have to remember to change each one of them, update anomaly. So, this is, these are the things which make, which tell us that 1NF is not enough, just having atomic values is not enough for this.

(Refer Slide Time: 10:46)

 IIT Madras
BSc Degree

Module 26
Partha Pratim Das
Week Recap
Objectives & Outline
Normal Forms
1NF
2NF
3NF
Module Summary

1NF (3): Possible Redundancy

- When LHS is not a Superkey :
 - Let $X \rightarrow Y$ be a non trivial FD over R with X is not a superkey of R , then redundancy exist between X and Y attribute set.
 - Hence in order to identify the redundancy, we need not to look at the actual data, it can be identified by given functional dependency.
 - Example : $X \rightarrow Y$ and X is not a Candidate Key
 $\Rightarrow X$ can duplicate
 $\Rightarrow$ Corresponding Y value would duplicate also.
- When LHS is a Super
 - If $X \rightarrow Y$ is a non trivial a superkey of R , then re exist between X and Y .
 - Example : $X \rightarrow Y$ and ,
 $\Rightarrow X$ cannot duplicate
 $\Rightarrow$ Corresponding Y value duplicate.

X	Y
1	3
1	3

X	Y
1	4
2	6
3	4

So, if we just, see look at this in the functional dependency perspective, then we see that if x determines y is a non-trivial functional dependency over R , where x is not a super key of R , then there will exist redundancy. Because, because x is not a super key, so, x can be duplicate, as you can see here x can be duplicate, x can be duplicate. And if x is, x determines y , so, if x is duplicate y can also be duplicate. But if it is a super key, then it will not be able to duplicate.

So, you will have this though, y can still duplicate because x determines y . So, obviously, y has no uniqueness here. So, these are things which, if they exist, will cause redundancy and the corresponding anomaly.

(Refer Slide Time: 11:48)

 IIT Madras
BSc Degree

Module 26
Partha Pratim Das
Week Recap
Objectives & Outline
Normal Forms
1NF
2NF
3NF
Module Summary

2NF: Second Normal Form

- Relation R is in Second Normal Form (2NF) only iff :
 - R is in 1NF and ✓
 - R contains no Partial Dependency

Partial Dependency:

Let R be a relational Schema and X, Y, A be the attribute sets over R where X is a Candidate Key, Y : Proper Subset of Candidate Key, and A : Non Prime Attribute

If $Y \rightarrow A$ exists in R , then R is not in 2NF.

$(Y \rightarrow A)$ is a Partial dependency only if

- Y : Proper subset of Candidate Key ✓

While, 1NF as you have seen, addresses some of the redundancy considerations, but it is not enough. We need to tighten it further. So, we look into what is known as a 2NF or second normal form. For a relation to be in second normal form, it must be in 1NF, first normal form, like normal forms are like a hierarchy of normal form. So, any higher normal form is by definition, will satisfy the conditions of the normal form below it. And it will have some additional condition.

So, obviously, a second normal form relation will have to be in first normal form, the multivalued issues resolved and so on. But in addition, it must not contain any partial dependency. So, let us understand what is a partial dependency. Let us say R be a relational schema and x is some candidate key of R , which could be the primary key also. But what is of importance is to consider y and A , where y is a proper subset of some candidate key, y is a proper subset of some candidate key.

And A is an attribute set of non-prime attribute. So, you would recall, we mentioned this in, in some of the problem solving, problem statements. So, what is a prime attribute? A prime attribute of a relation is an attribute, that is a part of some candidate key of the relation. If an attribute is part of some candidate key, then it is called a prime attribute. So, what is a non-prime attribute? A non-prime attribute is one, which is not a, which does not belong to any of the candidate keys of the relation.

So, given this if y is a proper subset of a candidate key and proper subset, you will have to remember. Proper subset of a candidate key and A is a non-prime attribute, then y determines A is a partial dependency. So, these are the two conditions we will have to remember. And if there is a partial dependency, then the relation is not in 2NF, it violates 2NF. So, for 2NF again, the relation will have to be in 1NF and there must be no partial dependency as we have defined. So, going next we will see some examples of this 2NF relations.

(Refer Slide Time: 14:55)

2NF (2)

Module 26
Partha Pratim Das
Week Recap
Objectives & Outline
Normal Forms
1NF
2NF
3NF
Module Summary

- STUDENT(Sid, Sname, Cname) (already in 1NF)

Students:

SID	Sname	Cname
S1	A	C
S1	A	C++
S2	B	C++
S2	B	DB
S3	A	DB

{SID, Cname}: Primary Key

Functional Dependencies:
 $\{SID, Cname\} \rightarrow Sname$
 $SID \rightarrow Sname$

Partial Dependencies:
 $SID \rightarrow Sname$ (as SID is a Proper Subset of Candidate Key $\{SID, Cname\}$)

Key Normalization

R1:

SID	Sname
S1	A
S2	B
S3	A

{SID}: Primary Key

The above two relations are:
1. Lossless Join
2. 2NF
3. Dependency Preserving

- Redundancy?
 - Sname
- Anomaly?
 - Yes

2NF (2)

Module 26
Partha Pratim Das
Week Recap
Objectives & Outline
Normal Forms
1NF
2NF
3NF
Module Summary

- STUDENT(Sid, Sname, Cname) (already in 1NF)

Students:

SID	Sname	Cname
S1	A	C
S1	A	C++
S2	B	C++
S2	B	DB
S3	A	DB

{SID, Cname}: Primary Key

Functional Dependencies:
 $\{SID, Cname\} \rightarrow Sname$
 $SID \rightarrow Sname$

Partial Dependencies:
 $SID \rightarrow Sname$ (as SID is a Proper Subset of Candidate Key $\{SID, Cname\}$)

Key Normalization

R1:

SID	Sname
S1	A
S2	B
S3	A

{SID}: Primary Key

The above two relations are:
1. Lossless Join
2. 2NF
3. Dependency Preserving

- Redundancy?
 - Sname
- Anomaly?
 - Yes

2NF (2)

Module 26
Partha Pratim Das
Week Recap
Objectives & Outline
Normal Forms
1NF
2NF
3NF
Module Summary

- STUDENT(Sid, Sname, Cname) (already in 1NF)

Students:

SID	Sname	Cname
S1	A	C
S1	A	C++
S2	B	C++
S2	B	DB
S3	A	DB

{SID, Cname}: Primary Key

Functional Dependencies:
 $\{SID, Cname\} \rightarrow Sname$
 $SID \rightarrow Sname$

Partial Dependencies:
 $SID \rightarrow Sname$ (as SID is a Proper Subset of Candidate Key $\{SID, Cname\}$)

Key Normalization

R1:

SID	Sname
S1	A
S2	B
S3	A

{SID}: Primary Key

$SID \rightarrow Sname$

The above two relations are:
1. Lossless Join
2. 2NF
3. Dependency Preserving

- Redundancy?
 - Sname
- Anomaly?
 - Yes

So, let us take a simple example. I have a student relationship, which has student ID, student name and course name, which is already in 1NF, we have seen, where you have on the diagram on right, on the diagram here, this is your you can see these are the FDs. That is student ID decides, student Sname, and student ID and course ID together is a key. Now, is there an anomaly? Obviously, there is anomaly, there is possibilities of anomaly.

Because you have multiple entries of the students name. So, these are the functional dependencies as you have written over. Of that if you see look at this SID, this one, this is a partial dependency. How do I say? SID is a subset of a key, first condition. And Sname is not a subset of, is not a member of any key. That is, it is a non-prime attribute. So, this is y as we wrote in earlier and this is A as we wrote earlier. So, this is a partial dependency.

Now, because of that, see, if the left hand side of this was a key, then what would have happened? It would not have been possible to have duplicates on that. And therefore, your Sname would not have been replicated number of times, the student's name would not have been replicated a number of times, but it is not. So, what is happening is SID has multiple instances, in a particular instance of the database. Because it comes with different Cname, giving it the uniqueness.

And for each instance of identical instances of SID, the Sname will get repeated. If Sname were a part of the key, it would not have happened because it would not have been possible to have multiple identical values of on it. So, that is, that is the partiality whole about. So, that is a two sides, people are very nicely tried to bind that on one side, on the left hand side, it is a subset of the of the key set. So, it can be occur in multiple numbers and on the right it is not a member of any key set, is a non-prime attribute, so you can replicate.

Now, obviously, this can be very easily handled by doing a normalization, what you do is simply put this together and you put this together. Just, you will have to have the common attributes SID. So, here on the left in R1, you do not have any redundancy in R2 either, you do not have any redundancy. SID is a key on the on R1. So, you can see that, an R1 intersection R2 is SID. So, you can easily see that the Lossless Join condition is satisfied.

And now the dependencies are, this dependency, your dependencies have now become this is a SID determining Sname. And this is SID Cname determining R2, that is both; both are required, trivial dependencies. So, given that your dependencies will get preserved. There is no case of partial dependency remaining. So, this is in 2NF. This is a Lossless Join and is dependency preserving. So, certainly this is a better design than what we had earlier.

(Refer Slide Time: 19:48)

2NF (3): Possible Redundancy

Module 26
Partha Pratim Das
Work Recap
Objectives & Outline
Normal Forms
(1)
2NF
(2)
(3)
Module Summary

● **Supplier(SID, Status, City, PID, Qty)**

Supplier:

SID	Status	City	PID	Qty
S1	30	Delhi	P1	100
S1	30	Delhi	P2	125
S1	30	Delhi	P3	200
S1	30	Delhi	P4	130
S2	10	Karnal	P1	115
S2	10	Karnal	P2	250
S3	40	Rohatak	P1	245
S4	30	Delhi	P4	300
S4	30	Delhi	P5	315

Partial Dependencies:
 $SID \rightarrow Status$
 $SID \rightarrow City$

Post Normalization

Sup, City:
[SID] Status City
FDD of Sup, City:
[SID] Status City

Drawbacks:

- **Deletion Anomaly:** If we delete a supplier, then we not only delete information about a supplier, but also the status value of a particular city.
- **Insertion Anomaly:** We cannot insert a new supplier until a supplier exists.

The supplier case again you can see. So, you can see that well with the supplier, these are the dependencies that we have. And therefore, I have partial dependency, in terms of say city. Because city is a non-prime attribute and the left hand side SID deciding city is a part of the key, not a key by itself. Similarly, the other one is also a partial dependency. So, I can easily decompose them by keeping this on one side and keeping this on the other side.

This is the decomposition and the redundancy that we had, is getting removed through this process. Now, the question is, does it still have some redundancy left? You will have to check for that information. Now, we have seen that, if we let me just go back once more. Now, if you, if you see we have not created the instance here, but if you see that SID decides the city. City decides the status and therefore, SID decides the status. So, what you will have here is

you have given the SID, which is the key. You will have a unique city and therefore, you will have a unique status. So, that is how basically your redundancy is managed in the 2NF here.

(Refer Slide Time: 22:13)

Module 26

Partha Pratim Das

Week Recap

Objectives & Outline

Normal Forms

1NF

2NF

3NF

Module Summary

3NF: Third Normal Form

Let R be the relational schema.

- [E. F. Codd, 1971] R is in 3NF only if:
 - R should be in 2NF
 - R should not contain transitive dependencies (OR, Every non-prime attribute is non-transitively dependent on every key of R)
- [Carlo Zaniolo, 1982] Alternately, R is in 3NF iff for each of its functional dependencies one of the following conditions holds:
 - X contains A (that is, A is a subset of X , meaning $X \rightarrow A$ is trivial functional dependency)
 - X is a superkey, or
 - Every element of $A - X$, the set difference between A and X , is a *prime attribute* in $A - X$ is contained in some candidate key)
- [Simple Statement] A relational schema R is in 3NF if for every FD $X \rightarrow A$ associated with R :
 - $A \subseteq X$ (that is, the FD is trivial) or
 - X is a superkey

Module 26

Partha Pratim Das

Week Recap

Objectives & Outline

Normal Forms

1NF

2NF

3NF

Module Summary

2NF (3): Possible Redundancy

- Supplier(SID, Status, City, PID, Qty)

SID	Status	City	PID	Qty
S1	30	Delhi	P1	100
S1	30	Delhi	P2	125
S1	30	Delhi	P3	200
S1	30	Delhi	P4	130
S2	10	Karnal	P1	115
S2	10	Karnal	P2	250
S3	40	Rohatak	P1	245
S4	30	Delhi	P4	300
S4	30	Delhi	P5	315

Partial Dependencies:
 $SID \rightarrow Status$
 $SID \rightarrow City$

Post Normalization

Sup, City: SID, Status, City

FDD of Sup, City:

Drawbacks:

- Deletion Anomaly:** If we delete a supplier, then we not only lose information about a supplier, but also the status value of a particular city.
- Insertion Anomaly:** We cannot insert a new status value until a supplier exists for that status.

Now, going forward, let us, the next is third normal form. If you have redundancy left, after the second normal form you go to the third normal form. Third normal form has evolved over a period of time, through independent work. So, the first two definitions which I will not go through is just more for reference. One in 72 and another is in 82, have actually defined third normal form but using different other notions.

The simple third normal form statement, which is accepted today is simply saying that, it is in second normal form and does not have a transitive dependency. This is the, this is a simple thing that we say. Now, putting it in, in detail is there is no trivial dependency. A subset of X ,

X has to be a super key or A has to be a part of the candidate key. Either of these has to happen, and with that we actually have that every 3NF relation is also in naturally in 2NF.

(Refer Slide Time: 23:37)

3NF (2): Transitive Dependency

- A **transitive dependency** is a functional dependency which holds by transitivity. A transitive dependency can occur only in a relation that attributes.
- Let A , B , and C designate three distinct attributes (or distinct collections) in the relation. Suppose all three of the following conditions hold:
 - $A \rightarrow B$
 - It is not the case that $B \rightarrow A$
 - $B \rightarrow C$
- Then the functional dependency $A \rightarrow C$ (which follows from 1 and 3 transitivity) is a transitive dependency

So, here we talk about transitive dependency. A transitive dependency is a functional dependency, which hold by virtue of transitivity. The transitivity can occur only in a relation that has three or more attributes. Because A determines B, B determines C. So, if we have three attributes A B and C or this could be distinct collections also, need not be just attributes. Then let us say that A determines B, B does not determine A and B determines C.

Then the functional dependency, A determine C which come from the first and the third by the transitivity of Armstrong's axiom is known as a functional, as a transitive dependency. Mind you, the second condition is important for transitive dependency because if A determines B and B determines C, then it necessarily means that these are equivalent attributes, they are the strongly tied together. So, it is not a case of, because then B determine C is also A determine C anyway. It is there is no, it is not happening just because of the transitivity.

(Refer Slide Time: 25:14)

3NF (3): Transitive Dependency

- Example of **transitive dependency**
- The functional dependency $\{Book\} \rightarrow \{Author\ Nationality\}$ applies; the book, we know the author's nationality. Furthermore:
 - $\{Book\} \rightarrow \{Author\}$
 - $\{Author\}$ does not $\rightarrow \{Book\}$
 - $\{Author\} \rightarrow \{Author\ Nationality\}$
- Therefore $\{Book\} \rightarrow \{Author\ Nationality\}$ is a transitive dependency
- Transitive dependency occurred because a non-key attribute (Author) another non-key attribute (Author Nationality).

Book	Genre	Author	Author Nationality
Twenty Thousand Leagues Under the Sea	Science Fiction	Jules Verne	French
Invaders from the Planet of the Apes	Science Fiction	Jules Verne	French

3NF (4): Example

- Example:
Sup.City(SID, Status, City) (already in 2NF)

SID	Status	City
S1	30	Delhi
S2	10	Karnal
S3	40	Rohtak
S4	30	Delhi

SID: Primary Key

Redundancy?
◦ Status

Functional Dependencies:
 $SID \rightarrow Status$,
 $SID \rightarrow City$,
 $City \rightarrow Status$

Transitive Dependency :

Sup.City:

SID	Status	City
S1	30	Delhi
S2	10	Karnal
S3	40	Rohtak
S4	30	Delhi

FDD of Sup.City:

The abc and CS

- Loss
- 3NF
- Dep

3NF (4): Example

- Example:
Sup.City(SID, Status, City) (already in 2NF)

SID	Status	City
S1	30	Delhi
S2	10	Karnal
S3	40	Rohtak
S4	30	Delhi

SID: Primary Key

Redundancy?
◦ Status

Functional Dependencies:
 $SID \rightarrow Status$,
 $SID \rightarrow City$,
 $City \rightarrow Status$

Transitive Dependency :

Sup.City:

SID	Status	City
S1	30	Delhi
S2	10	Karnal
S3	40	Rohtak
S4	30	Delhi

FDD of Sup.City:

The abc and CS

- Loss
- 3NF
- Dep

So, an example say you have a functional dependency that book determines author nationality. Assuming that the books have one author, otherwise it will not hold it. Book determines author and the author's nationality. So, what we can say is book determines author, naturally. Author, does not determine book, because the author may have written multiple books. Author does not determine book. But author determines author nationality.

The author will determine the nationality, obviously. So, book determines author nationality is a transitive dependency. So, this transitive dependency, why did it occur? Because there is a non key attribute author, which has been determined from, which is determining another non key attribute, author nationality. One non key attribute author is determining another non key attribute, author nationality. And that is where the basic problem of the transitive application happens.

So, if we get back to the earlier situation, earlier example, this is one and this is the other, two functional dependencies. Now, that leads us to city, SID determining status, which is this transitive dependency. Because you do not have, you have SID determining city, but you do not have city determining SID. Naturally, multiple suppliers can be there in the same city. So, there is possibility of anomalies here. So, what we will do is, we will normalize say based on, we will break this transitivity.

So, how do we do that? We do SID city and we do city status, put them in separate buckets. As you do that, SID is a key of the SC component and city is the key of the CS component. CS is the common attribute, is a key of CS. So, city will ensure that the join is lossless. These are in three normal form, because it is in two normal form and there is no transitive dependency. And you can easily see that this preserve the dependency, because SC can check for SID determining city. CS can check for city determining status, and therefore by closure SID determining status will get checked as well.

(Refer Slide Time: 28:27)

3NF (5): Example

- Relation *dept_advisor*(*s_ID*, *i_ID*, *dept_name*)
- $F = \{s_ID, dept_name \rightarrow i_ID, i_ID \rightarrow dept_name\}$
- Two candidate keys: *s_ID*, *dept_name*, and *i_ID*, *s_ID*
- *R* is in 3NF
 - $s_ID, dept_name \rightarrow i_ID$ ✓
 - ▷ *s_ID*, *dept_name* is a superkey
 - $i_ID \rightarrow dept_name$ ✓
 - ▷ *dept_name* is contained in a candidate key

A relational schema *R* is in 3NF if for every FD $X \rightarrow A$ associated with *R* either

- $A \subseteq X$ (i.e., the FD is trivial) or
- *X* is a superkey of *R* or

Another a relation department advisor. So, you have sID student name, instructor name and department name, where student ID and department name decides instructor which is obvious. And the instructor name, decides the department name. So, there are two candidate keys for this relation. One is sID and department name, because it determines iID. And sID and iID because that will determine the department name as well. So, there are two candidate keys.

Now is this relation in 3NF, let us check the dependencies. sID, department name decides iID, sID department name, department iID. So, sID department name is a super key, no problem. So, this is fine. iID chains determines department name. But here iID is not a super key, but department name is a part of a candidate key, part of the candidate key here. So, this relation is in third normal form. And we are happy to go with it, as a third normal form relation.

(Refer Slide Time: 29:53)

3NF (6): Redundancy

- **There is some redundancy in this schema**
- Example of problems due to redundancy in 3NF ($J : s_ID, L : i_ID, K$)
 - $R = (J, L, K). F = \{JK \rightarrow L, L \rightarrow K\}$

J	L	K
j_1	l_1	k_1
j_2	l_1	k_1
j_3	l_1	k_1
null	l_2	k_2

- Repetition of information (for example, the relationship l_1, k_1)
 - ($i_ID, dept_name$)

3NF (6): Redundancy

- **There is some redundancy in this schema**
- Example of problems due to redundancy in 3NF ($J : s_ID, L : i_ID, K$)
 - $R = (J, L, K). F = \{JK \rightarrow L, L \rightarrow K\}$

J	L	K
j_1	l_1	k_1
j_2	l_1	k_1
j_3	l_1	k_1
null	l_2	k_2

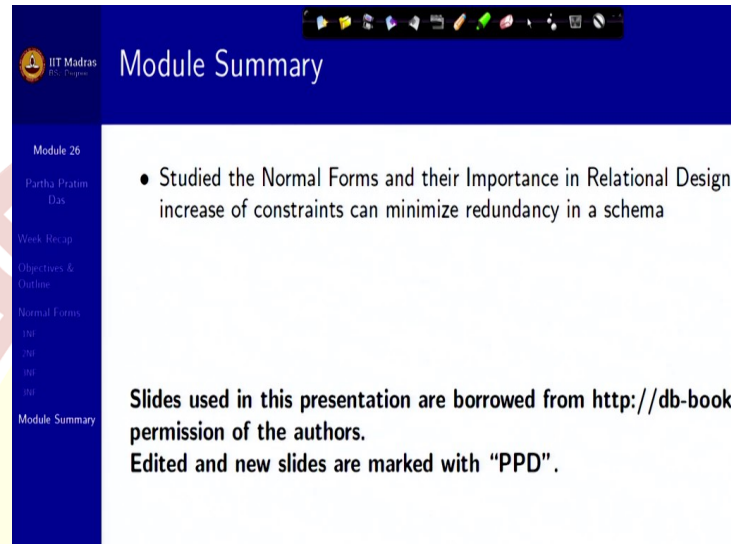
- Repetition of information (for example, the relationship l_1, k_1)
 - ($i_ID, dept_name$)

Now, before we close, let me just show you that, this third normal form relation may also have redundancy. I mean just to make the writing simpler, I use some coding J for sID, L for iID and K for department name. So, it, this is how it looks like, sID and department names determines iID, JK determines L and L determines K iID determines the department name. And these are different entries, which satisfy, you can see that there is no entry, where J and L are same, and K is different.

So, that is satisfied, L determines K. So, whenever I have l_1 , I have this value as k_1 . So, this is fine. Now, there is of course, this relation is in third normal form, but there is repetition of information as you can see here. And suppose, I there is an instructor, who is not supervising any student right now. So, that is sID. So, but that instructor needs to have the department

name. So, if I have, I2, whose department is K2, but there is no sID, there is no student being supervised, then I have to use null values to, for this. So, well, this is as we said, this is came in third normal form, but still there are problems that will keep on hitting us, we will have to do more in terms of them.

(Refer Slide Time: 32:01)



IIT Madras
R2V Program

Module Summary

Module 26
Partha Pratim Das

- Studied the Normal Forms and their Importance in Relational Design
increase of constraints can minimize redundancy in a schema

Slides used in this presentation are borrowed from <http://db-book>
permission of the authors.
Edited and new slides are marked with "PPD".

So, in this module, we have introduced the basic notions of three critical normal forms. Particularly, the third normal form. Because, it is often, very often that it is not enough to just to do the first or second normal form. And we will see subsequently, that there is further reasons to focus more on third normal form. Because, as we had seen earlier, that decomposing a relation in Boyce-codd normal form, though it guarantees Lossless Join, it cannot guarantee dependency preservation, I will, shown examples of that.

But for third normal form, we will show an algorithm, we will show a way so that any relation can be decomposed into third normal form, which is both Lossless Join as well as preserves the dependency. The other reality of third normal form being very prevalent is the fact that, the subsequent normal forms are quite more difficult to model, identify the requirements through multivalued dependencies and so on that we are going to talk of in the real world.

And as it happens that, most of the situations that arise or a large number of the situations that arise in the real world, gets very well addressed with the third normal form. So, we will have to focus some more on the third normal form. Remember that, a third normal form relation is in a relation which is in second normal form, but does not have partial transitive dependency. A second normal form is in first normal form, which does not have partial dependency.

So, third normal form is atomic, does not have partial dependency and does not have transitive dependency. With those, let us close the module here today. And thank you very much for your attention. We will continue this in the next module.

